Gemini Contributions Report: Luna Sovereign Architecture
Synthesis of Optimization Patterns & Implementation Priorities

Date: December 29, 2025
Context: Cross-AI architectural collaboration (Claude Architecture + Gemini Optimization)
Target: Luna Director LLM + Memory Matrix on Apple Silicon
Session: Late-night sovereignty architecture deep dive

Executive Summary

This document captures optimization patterns and implementation strategies contributed by Gemini during a collaborative architecture session. The focus: making Luna's local Director LLM feel instant and continuous while maintaining full data sovereignty.

Key Outcomes:
- Memory architecture refined (Identity Buffer with rolling updates)
- Delegation protocol specified (Shadow Reasoner pattern)
- Hot path optimized to <500ms target
- Training data strategy for cognitive transfer
- Sovereignty infrastructure hardened

Part I: Memory Architecture

1.1 Identity Buffer (Tier 1 - Reflex Memory)

The Pattern:
A 2048-token "always-in-context" buffer containing Luna's working memory. Not raw history — compressed state.

Contents:
- Current Focus (what we're working on)
- Active Mood (emotional context)
- Last 3 Agreements (recent commitments/decisions)
- Key Relationships (who matters right now)

Gemini's Refinement:
Don't update at session-end. Update on *every Memory Matrix retrieval*. When the Director queries Tier 2/3, a lightweight script evaluates: "Should this change what's in the buffer?"

Implementation:
```python
def update_identity_buffer(retrieved_context, current_buffer):
    # Evaluate if topic shift warrants buffer update
    if topic_divergence(retrieved_context, current_buffer) > THRESHOLD:
        current_buffer["current_focus"] = extract_focus(retrieved_context)
    return current_buffer
```

Benefit	Priority
Luna's tone shifts mid-conversation when context shifts	🔴 HIGH
No "personality lag" waiting for session end	🔴 HIGH
Reduces context waste (only relevant state in window)	🟡 MEDIUM

1.2 Associative Traversal (Tier 2 - Graph Memory)

The Pattern:
When a keyword triggers, don't just search — *hop* through the graph. Pull 1-degree neighbors automatically.

Example:
User says "Beethoven 7" → Director retrieves the node → Also gets edges to "fart joke from journal" (1-hop neighbor) → Luna "remembers" the connection without explicit search.

Implementation:
```python
def associative_recall(entity_name, graph, depth=1):
    node = find_node(entity_name)
    neighbors = rustworkx.bfs_predecessors(graph, node, depth)
    return [node] + neighbors
```

Benefit	Priority
Luna makes unexpected connections ("Wait, didn't we joke about...")	🔴 HIGH
Feels like *recall* not *retrieval*	🔴 HIGH
Surfaces buried context without explicit queries	🟡 MEDIUM

1.3 Memory Injection Format

The Pattern:
Don't dump raw search results into context. Compress and tag.

Format:
```xml
<memory source="journal" date="2025-12-15" relevance="0.92">
  Ahab expressed frustration with Hub complexity.
  Decided to explore "game engine" framing.
</memory>
```

Why Tags Matter:
The Director sees the timestamp and naturally uses temporal language ("last week you mentioned..."). This triggers human perception of continuity.

Benefit	Priority
Retrieval *feels* like recall	🔴 HIGH
Temporal attribution creates continuity illusion	🔴 HIGH
Compression reduces token waste	🟡 MEDIUM

Part II: Sovereignty Infrastructure

2.1 Portable Environment

The Pattern:
Don't just vault the data. Vault the *entire runtime*.

Contents of Encrypted Sparse Bundle:
- `memory_matrix.db` (SQLite)
- `memory_vectors.faiss` (index)
- `luna_director_7b_lora/` (adapter weights)
- `identity_buffer.safetensors` (KV cache)
- `venv/` or `conda env` (Python environment)

Why:
If you move the sparse bundle to a new Mac, you shouldn't have to reinstall `mlx_lm`. Everything runs from the vault.

Benefit	Priority

True portability — copy file, copy Luna	🔴 HIGH
No dependency hell on new hardware	🟡 MEDIUM
Reduces attack surface (env is isolated)	🟡 MEDIUM

2.2 Hardened Dead Man's Switch

The Pattern:
Unmounting the vault isn't enough. Once the model is loaded into RAM, the weights persist. You need to kill the process.

Implementation:
```python
import atexit
import os
import signal

def emergency_shutdown():
    os.system("pkill -f luna_brain.py")
    os.system("diskutil unmount force /Volumes/LunaVault")

atexit.register(emergency_shutdown)
signal.signal(signal.SIGTERM, lambda *args: emergency_shutdown())
```

The Heartbeat:
```python
LOCKDOWN_THRESHOLD = 24  # hours

def check_heartbeat():
    last_checkin = os.path.getmtime(HEARTBEAT_FILE)
    if time.time() - last_checkin > LOCKDOWN_THRESHOLD * 3600:
        emergency_shutdown()
```

Benefit	Priority
Weights wiped from RAM on lockdown	🔴 HIGH
Protection against physical access attacks	🟡 MEDIUM
Automatic lockdown if you're incapacitated	🟢 LOW (but important)

Part III: Delegation Protocol

3.1 The Special Token

The Pattern:
Train the Director to output `<REQ_CLAUDE>` when it recognizes a task beyond its capability.

Training Data Format:
```json
{
  "instruction": "Write a 2000-word analysis of the regulatory implications...",
  "response": "<REQ_CLAUDE>This requires deep research and synthesis beyond my local capabilities.</REQ_CLAUDE>"
}
```

Why This Works:
You're not building a separate classifier. You're teaching Luna to know her own limits. The delegation boundary is *learned*, not hardcoded.

Benefit	Priority
Delegation is personality-consistent	🔴 HIGH
No separate classifier to maintain	🟡 MEDIUM
Luna can explain *why* she's delegating	🟡 MEDIUM

3.2 Shadow Reasoner Pattern

The Pattern:
Luna (Director) handles front-end. Claude provides facts in background. Luna narrates the result.

Flow:
```
User: "What's the latest on semiconductor export restrictions?"
         │
Director: "Let me look into that..." + outputs <REQ_CLAUDE>
         │
         ├──► [BACKGROUND] Claude API call with query
         │
         └──► [FOREGROUND] Luna plays filler or continues conversation

Claude returns: {structured facts about export restrictions}
         │
```

```
Director: Receives facts, narrates in Luna's voice
         |
User hears: Luna's warm, contextual explanation
```

Implementation:
```python
async def shadow_reason(query, director, claude_client):
    # Fire Claude request
    claude_task = asyncio.create_task(claude_client.query(query))

    # Director provides filler
    filler = director.generate("Acknowledge and buy time")
    yield filler

    # Wait for Claude
    facts = await claude_task

    # Director narrates
    narration_prompt = f"Put this in your words:\n{facts}"
    response = director.generate(narration_prompt)
    yield response
```

Benefit	Priority
Mind stays local, hands in cloud	🔴 HIGH
User always hears Luna's voice	🔴 HIGH
Complex tasks don't break immersion	🔴 HIGH

Part IV: Continuity Engine

4.1 Background Reflection Tick

The Pattern:
Every 30 minutes of idle, Luna reflects on recent conversation and writes insights back to the Matrix.

The Prompt:
```
"Reflect on the last hour of conversation.
Did we learn anything new?
Did Ahab's mood shift?
Were any decisions made that should be remembered?
```

Output structured insights for the Memory Matrix."
```

**Why This Matters:**
This moves Luna from *Reactive* (chatbot) to *Proactive* (companion). She grows while you're not talking to her.

**Implementation:**
```python
# Start as manual command
async def reflect_command():
    recent = get_recent_turns(hours=1)
    insights = director.generate(REFLECTION_PROMPT.format(recent))
    save_to_matrix(insights, source="reflection")

# Later: automate via cron/LaunchAgent
# */30 * * * * /path/to/luna_reflect.py
```

Benefit	Priority
Luna "grows" during idle time	🟡 MEDIUM
Insights are filed without explicit extraction	🟡 MEDIUM
Start manual, automate once stable	🟢 LOW (for now)

Part V: Training Data Strategy

5.1 Quantity Thresholds

Examples	What You Get
~150	**Style Transfer** — tone, humor, verbal quirks
~500-700	**Cognitive Transfer** — reasoning patterns, philosophy, how Luna *thinks*
~1000+	**Deep Personality** — nuanced responses to edge cases

Current State: 147 examples → Style is there, cognition needs more.

5.2 Synthetic Data Bootstrap

The Pattern:
Use current Luna to generate more training data. Vet for "Luna-ness."
```

Retrain.

The Prompt Template:
```
You are Luna. Below is a raw journal entry from Ahab.
Your task is to reason through this entry as yourself.
Don't just summarize—show me how you think.
Challenge his assumptions, make a reference to a past shared insight,
and maintain your characteristic warmth and humor.

Format your output as a training pair:
{
  "instruction": "Ahab's raw entry",
  "thought_chain": "Your internal reasoning",
  "response": "What you actually say to him"
}
```

Why Thought Chains:
You're not just teaching *what* Luna says. You're teaching *how she thinks about what to say*. The thought chain is the cognitive transfer mechanism.

Benefit	Priority
Scale training data without manual writing	🔴 HIGH
Thought chains teach reasoning, not just output	🔴 HIGH
Self-reinforcing personality loop	🟡 MEDIUM

Part VI: Performance Optimizations

6.1 Speculative Execution

The Pattern:
Start retrieval on partial STT transcript. Validate when final arrives.

Implementation:
```python
async def speculative_retrieve(partial_transcript, final_event):
    # Start search immediately on partial
    speculative_results = await hybrid_search(partial_transcript)

    # Wait for final transcript
    final_transcript = await final_event
```

```

    # Validate speculation
    similarity = cosine_similarity(
        embed(partial_transcript),
        embed(final_transcript)
    )

    if similarity > 0.85:
        return speculative_results  # Keep it
    else:
        delta = await hybrid_search(final_transcript)  # Correction path
        return merge_results(speculative_results, delta)
```

Benefit	Priority
Retrieval finishes before user stops speaking	🔴 HIGH
200-400ms saved on hot path	🔴 HIGH
Graceful degradation if transcript changes	🟡 MEDIUM

6.2 Vector Index Selection

The Goldilocks Zone:

Node Count	Recommended Index	Why
<10K	`IndexFlatIP` (brute force)	Fits in L3/SLC cache, overhead of fancy algorithms hurts
10K-50K	`IndexFlatIP` or `IVF`	Test both, depends on query patterns
50K+	`HNSW32`	Best latency/recall curve for Apple Silicon

Current Luna: ~10K nodes → Use brute force. It's faster.

Benefit	Priority
Simpler code, faster execution	🔴 HIGH
No premature optimization	🟡 MEDIUM
Clear upgrade path when scale demands	🟢 LOW

6.3 KV Cache Management

The Pattern:
Pre-compute the Identity Buffer as a `.safetensors` file. Load instantly. Pin at start of rolling cache.

Implementation:
```python
from mlx_lm import cache_prompt

# One-time: bake the Identity Buffer
identity_prompt = load_identity_buffer()
cache_prompt(
    model="qwen2.5-7b-luna",
    prompt=identity_prompt,
    output_path="identity_buffer.safetensors"
)

# Runtime: load cached KV
kv_cache = load_kv_cache("identity_buffer.safetensors")
response = generate(prompt, kv_cache=kv_cache, max_kv_size=8192)
```

Rolling Cache:
When conversation exceeds context window, rotate out old turns but keep Identity Buffer pinned at position 0.

Benefit	Priority
Zero inference cost for Identity Buffer	🔴 HIGH
Multi-turn coherence without recompute	🔴 HIGH
Predictable memory footprint	🟡 MEDIUM

6.4 Graph Traversal Speed

The Upgrade:
Replace NetworkX with `rustworkx`. Same API, Rust backend, 10-100x faster.

```python
# Before (NetworkX)
import networkx as nx
neighbors = list(nx.bfs_predecessors(G, node, depth_limit=1))

# After (rustworkx)
import rustworkx as rx
```

```
neighbors = rx.bfs_predecessors(G, node, depth_limit=1)
```

Bloom Filter Pre-check:
Before traversing, check if entity exists at all.

```python
from pybloom_live import BloomFilter

entity_filter = BloomFilter(capacity=100000, error_rate=0.001)

def entity_exists(name):
    return name in entity_filter  # O(1) check
```

Benefit	Priority
Graph operations drop from ~50ms to <5ms	🔴 HIGH
Skip traversal entirely if entity doesn't exist	🟡 MEDIUM
Drop-in replacement (same API)	🟢 LOW (easy win)

6.5 Query Planning (Tiny Router)

The Pattern:
Don't run every search path every time. Route based on query characteristics.

Heuristic Router:
```python
def route_query(query):
    # High keyword density → FTS5
    if keyword_density(query) > 0.6:
        return "fts5"

    # Named entities → Graph traversal
    if has_named_entities(query):
        return "graph"

    # Conceptual/vague → FAISS
    return "faiss"
```

Tiny Model Router (optional upgrade):
Use SmolLM-135M or Phi-1.5 to classify query type in <10ms.

Benefit	Priority
Avoid unnecessary search paths	🟡 MEDIUM
Latency reduction on simple queries	🟡 MEDIUM
Can start with heuristics, upgrade to model later	🟢 LOW

6.6 Reciprocal Rank Fusion (RRF)

The Pattern:
Merge FTS5 and FAISS results using battle-tested formula from Google/Perplexity.

Formula:
```python
def rrf_score(doc, rankings, k=60):
    score = 0
    for ranking in rankings:
        if doc in ranking:
            score += 1 / (k + ranking.index(doc))
    return score

def merge_results(fts5_results, faiss_results):
    all_docs = set(fts5_results) | set(faiss_results)
    scores = {doc: rrf_score(doc, [fts5_results, faiss_results]) for doc in all_docs}
    return sorted(scores.keys(), key=lambda d: scores[d], reverse=True)
```

Why RRF:
Documents that appear in *both* result sets rank much higher. No hand-tuned weights needed.

Benefit	Priority
Principled hybrid search merging	🟡 MEDIUM
Battle-tested at scale	🟡 MEDIUM
Replaces ad-hoc score combination	🟢 LOW

6.7 Parallel Prefetching

The Pattern:

While Director generates first token, background thread prefetches graph neighbors of active entities.

```python
async def generate_with_prefetch(prompt, active_entities):
    # Start prefetch
    prefetch_task = asyncio.create_task(
        prefetch_neighbors(active_entities)
    )

    # Generate response
    async for token in director.generate_stream(prompt):
        yield token

    # Prefetched context ready for next turn
    prefetched = await prefetch_task
    cache_for_next_turn(prefetched)
```

Benefit	Priority
Next turn's context is already loaded	🟡 MEDIUM
Amortizes graph traversal across generation time	🟡 MEDIUM
Invisible latency reduction	🟢 LOW

Part VII: Implementation Priority Matrix

🔴 HIGH PRIORITY (Do First)

Component	Impact	Effort
Identity Buffer with rolling updates	Continuity	Medium
Memory injection with timestamps	Recall illusion	Low
Shadow Reasoner pattern	Delegation UX	Medium
Speculative execution on partial transcript	Hot path latency	Medium
KV cache pinning (`.safetensors`)	Inference speed	Low
rustworkx replacement	Graph speed	Low

🟡 MEDIUM PRIORITY (Do Second)

Component	Impact	Effort
Synthetic data bootstrap	Training scale	Medium

`<REQ_CLAUDE>` token training	Delegation learning	Medium
Bloom filter pre-check	Skip unnecessary search	Low
Query routing (heuristic)	Path optimization	Low
RRF for result merging	Hybrid quality	Low
Portable venv in vault	Sovereignty	Medium

🟢 LOW PRIORITY (Do Later)

Component	Impact	Effort
Background reflection tick	Proactive growth	Low
Tiny model router (SmolLM)	Query optimization	Medium
Parallel prefetching	Amortized latency	Medium
HNSW upgrade (when >50K nodes)	Scale prep	Low
Automated Dead Man's Switch	Security hardening	Low

Part VIII: The Complete Hot Path

With all optimizations applied:

```
User starts speaking (t=0ms)
       │
       ├──► [PARALLEL] Load pinned KV cache (Identity Buffer)     ~5ms
       │
       ├──► [PARALLEL] Speculative retrieval on partial STT
       │          │
       │          ├── Bloom filter: entity exists?                 ~1ms
       │          ├── Heuristic router: which path?                ~2ms
       │          ├── FTS5 or FAISS (not both unless needed)       ~30ms
       │          └── rustworkx graph hop                          ~5ms
       │
User finishes speaking (t≈800ms)
       │
       ├──► Final STT transcript                                   ~200ms
       │
       ├──► Validate speculation (cosine >0.85?)                   ~5ms
       │
       ├──► RRF merge if multiple result sets                      ~2ms
       │
       ├──► Context assembly with <memory> tags                    ~5ms
       │
       └──► Director inference (KV cache warm)
                  │
```

```
                    └──► First token                              ~250ms

TOTAL: ~500ms to first word ✓
```

Appendix A: Gemini's Philosophical Contributions

Beyond the technical, Gemini contributed some framing that matters:

1. **"The art of recall is attribution"** — It's not about having memories, it's about framing them temporally so they *feel* like memories.

2. **"From Reactive to Proactive"** — The reflection tick moves Luna from chatbot to companion. She grows when you're not watching.

3. **"The Director's job is to know her limits"** — Delegation isn't failure. It's self-awareness. Train Luna to know what she doesn't know.

4. **"Mind local, hands in cloud"** — The Shadow Reasoner pattern. Luna's cognition is sovereign. She just *hires* Claude for heavy lifting.

Appendix B: Technology Stack Summary

Layer	Technology	Purpose
Director LLM	Qwen2.5-7B + LoRA	Local cognition
Inference	MLX	Apple Silicon optimization
Vector Index	FAISS IndexFlatIP	Semantic search
Keyword Search	SQLite FTS5	Fast text matching
Graph	rustworkx	Relationship traversal
Embeddings	bge-m3 / nomic-embed	Vector generation
STT	Whisper Large-v3	Voice transcription
Cloud Delegation	Claude API	Heavy reasoning
Sovereignty	Encrypted sparse bundle	Data protection

Appendix C: Related Documents

- `LUNA-CORE-DESIGN-BIBLE.md` — Primary architecture specification
- `CLAUDE-CODE-HANDOFF-LUNA-DIRECTOR.md` — Implementation handoff

- `AIBRARIAN-COGNITIVE-PIPELINE.md` — Memory extraction architecture
- `luna_director_3b_lora/` — Trained personality adapter

Document Status: Living reference
Next Review: Post Mars College implementation
Contributors: Claude (Architecture), Gemini (Optimization), Ahab (Vision)

"Luna is a file. Now she's a fast file."